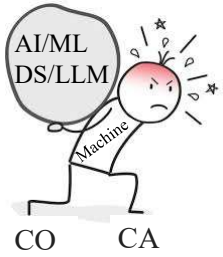
CS-2710-1 Spring 2026

# Computer Organization and Systems (COS)




## Lecture #16

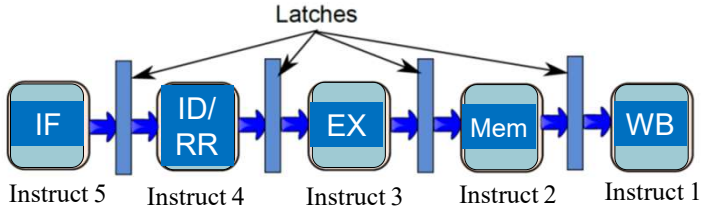
04 April 2026

Ashoka University

BBB

## MIPS Pipeline

- Pipeline registers (latches) are inserted between two consecutive pipeline stages
- All the registers are synchronized with the system clock and seamlessly transfer instruction-bits and other intermediate data forward, across consecutive pipeline stages




Status during the fifth clock pulse

## Data Hazards: Review

Data hazards may occur when we have:

instruction A

*followed by*

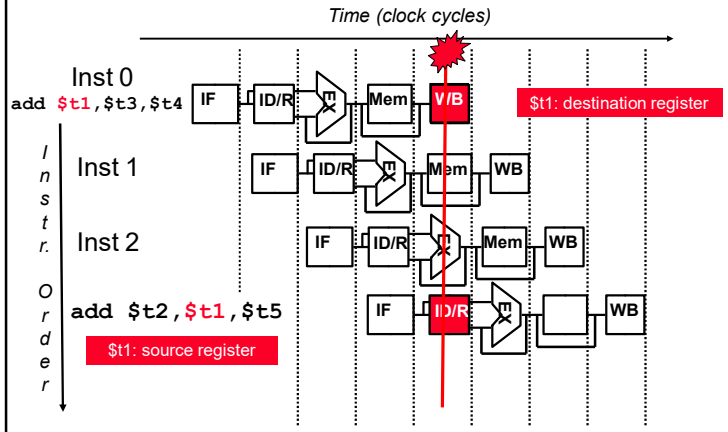
instruction B

*and*

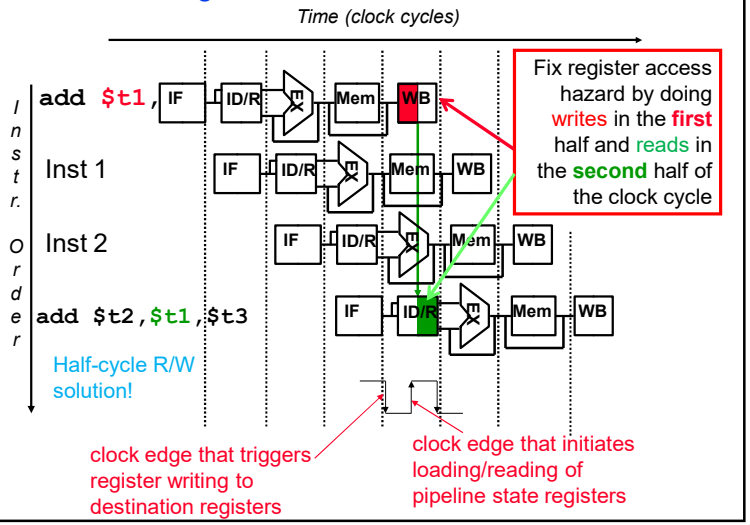
B wants to manipulate data (read/write) before A finishes ⇒ data hazard

Unless the pipeline halts or it takes some other corrective measure, computation will be erroneous

## Example: Hazards due to Register File Access

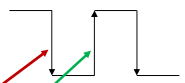


## Solution: Register Write-Read Hazard?




## Example: Half-Cycle Register Read/Write (to mitigate data hazard)

```
add $t1,$t3,$t4
lw ...
sw ...
add $t2,$t1,$t5
```



Hardware solution:
Perform register writes in the **first** half of the clock cycle;
Then a read for the same register (holding the updated data) for a subsequent instruction, can be made in the **second** half of the same clock cycle;

## Illustration:

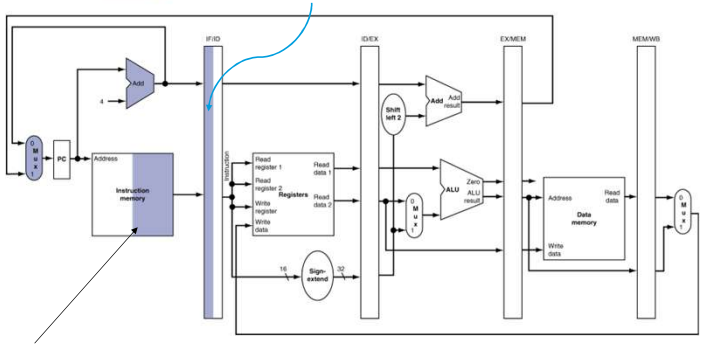
How *lw* instruction is executed in MIPS five-stage pipeline

IF → ID/RR → EX → Mem → WB

## IF for Load …

lw $t2, 0($t1)

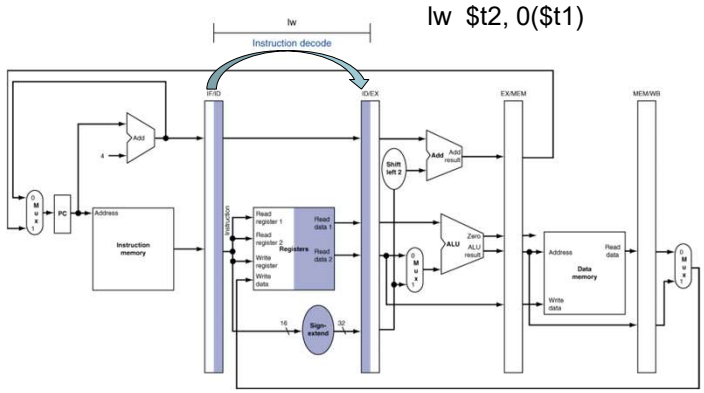
Information about the previous instruction that entered into the pipeline is being written into the IF/ID-latches in the first half of the first clock cycle, to be processed in the ID/RR-stage




IM is read to retrieve the current *lw* instruction, in the second-half of the first clock cycle, and put into a buffer

# ID/RR for Load…

lw $t2, 0($t1)

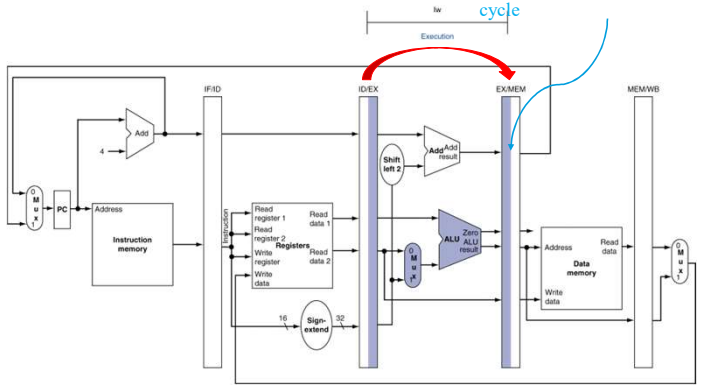





# EX for Load

lw $t2, 0($t1)

Address(Mem) = {0 + ($t1)} is copied in the EX/MEM latches along with IDs of $t2, $t1, in the third clock cycle

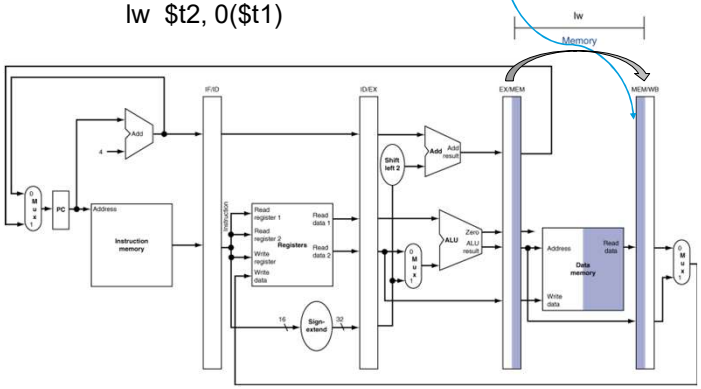





# MEM for Load

Mem(0 + $t1) is copied in the MEM/WB latches along with IDs of $t2, $t1;

lw $t2, 0($t1)

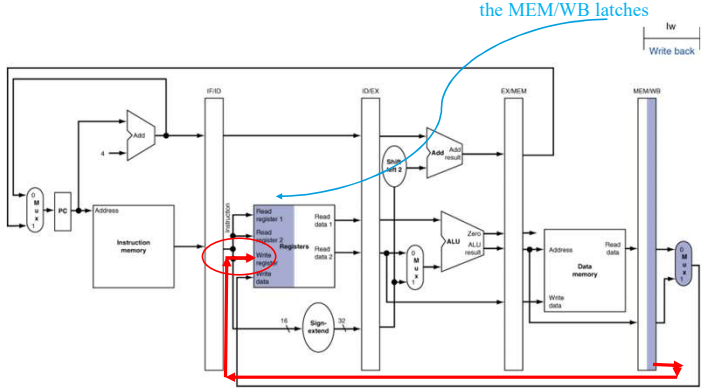




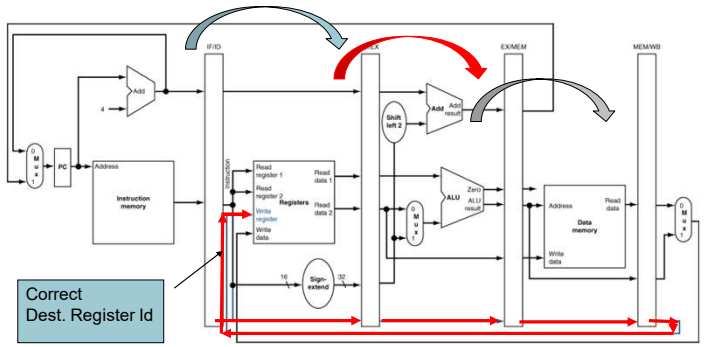

# WB for Load

lw $t2, 0($t1)

In the fifth clock cycle, data is written to the destination register $t2 (both the data and the ID of $t2 are provided by the MEM/WB latches



# Corrected Datapath for Load






# Mitigation of Data Hazards

```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

$s0 updated in the 5th clock cycle;

| cycle | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| add | IF | ID/RR | EX | MEM | WB | |
| sub | | IF | ID/RR | EX | MEM | WB |

half-cycle read/write

$s0 read request in 3rd clock cycle

Solution

| cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| add | IF | ID/RR | EX | MEM | WB | | | | |
| sub | | ✸ | ✸ | ✸ | IF | ID/RR | EX | MEM | WB |

reduces one stall ← IF ID/RR EX MEM WB

✸ Bubble (Stall): Pipeline interlocking

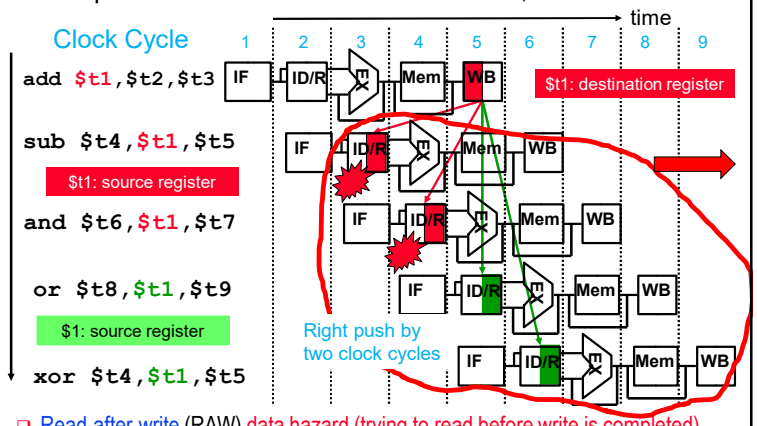
CPU-time increases; Can we do better??

# Pipelining: Data Hazards

- A *hazard* refers to the possibility of erroneous execution of an instruction in a pipeline
- A *data hazard* represents the possibility of erroneous execution because of the unavailability of correct data

# Data Hazard Example: Write/Read problem in a register

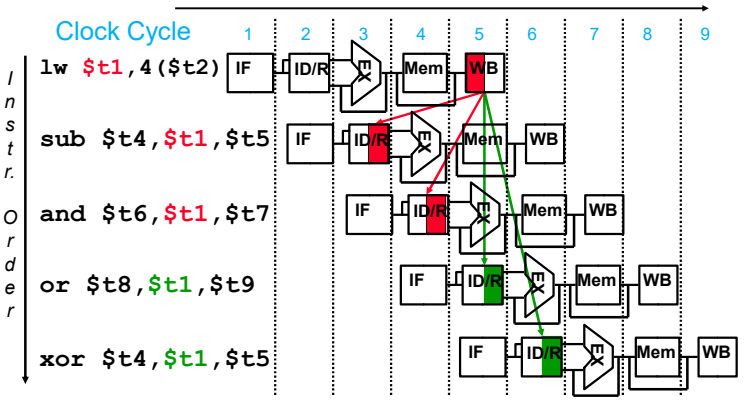
- Dependencies that are *backward* in time, cause hazards




- Read-after-write (RAW) data hazard (trying to read before write is completed)
- How many bubbles (stalls) are needed to resolve this hazard? Two bubbles!

## Loads Can Cause Data Hazards

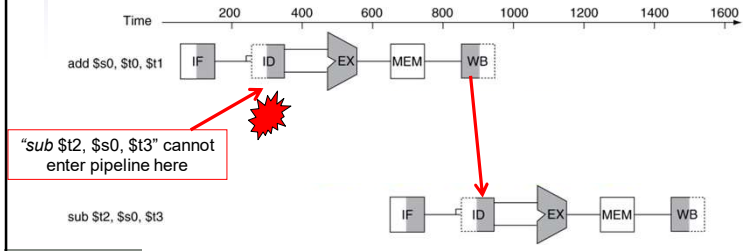
- Dependencies backward in time cause hazards




- Load-use data hazard

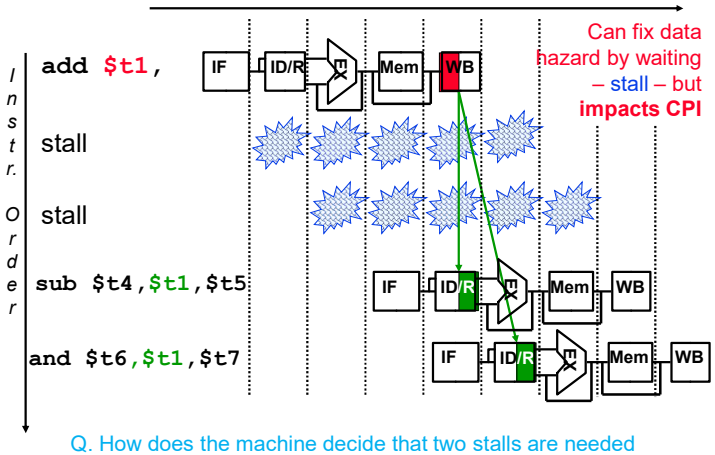
## Data Hazards (contd..)

- An instruction depends on completion of data access by a previous instruction
  - `add $s0, $t0, $t1`
    `sub $t2, $s0, $t3`



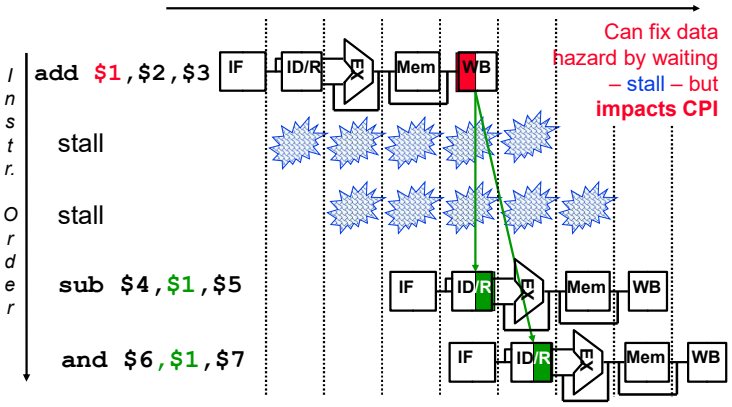



## One Way to "Fix" Data Hazard: Use Bubbles




Q. How does the machine decide that two stalls are needed and how it could be implemented?

## "Fixing" Data Hazard: Use Bubbles – Hardware Solution




A pipeline bubble is a NOP instruction that is inserted dynamically in a pipeline register by the **interlock hardware**. A bubble propagates through the pipeline just like normal instructions

## Pipelining: Data Hazards

```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

$s0 updated in the 5th clock cycle;

| cycle | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| add | IF | ID/RR | EX | MEM | WB | |
| sub | | IF | ID/RR | EX | MEM | WB |

$s0 read request in 3rd clock cycle

Solution

| cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| add | IF | ID/RR | EX | MEM | WB | | | | |
| sub | | ✸ | ✸ | ✸ | IF | ID/RR | EX | MEM | WB |

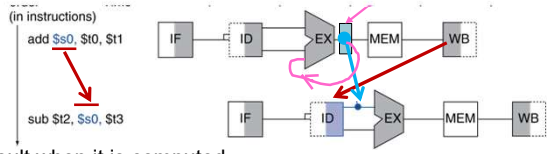
✸ Bubble (Stall): Pipeline interlocking

CPU-time increases; Can we do better??

## Improved hardware solution to fix data hazard: Forwarding (*a.k.a.* By-passing)

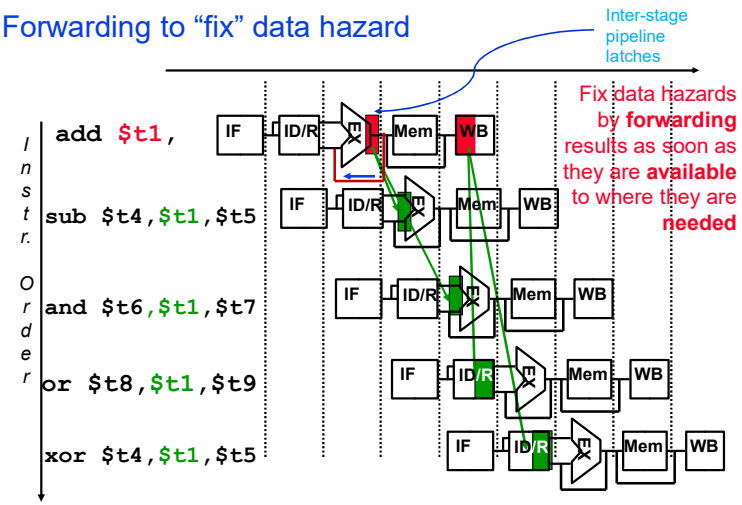
```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

| cycle | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| add | IF | ID/RR | EX | MEM | WB | |
| sub | | IF | ID/RR | EX | MEM | WB |




- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Additional datapath needed from the output of ALU (EX) to the input of the same ALU!

## Forwarding to "fix" data hazard




Fix data hazards by **forwarding** results as soon as they are **available** to where they are **needed**

Data forwarding (in time); data-path set backward (in space);
No stalls are needed (saving two)!

## Example: Data Hazards in Pipeline

| CC | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| ADD R1, R2, R3 | IF | ID | EX | MEM | WB | | | |
| SUB R4, R1, R5 | | IF | ID | EX | MEM | WB | | |
| AND R6, R1, R7 | | | IF | ID | EX | MEM | WB | |
| OR R8, R1, R9 | | | | IF | ID | EX | MEM | WB |

data hazard

With forwarding mechanism such as EX → EX and Mem → EX all data hazards in this cases can be eliminated.

Total # clock cycles to run this code with forwarding = 8
Suppose, there is no data forwarding; then # clock cycles = 10